

The code does not require any explanation, as it is similar to the directed graphs we generated in the previous article of this series. The key difference is that there are no directed cycles in this graph. Upon executing the program 'dacyclic16.sagews' using CoCalc, we obtain the directed acyclic graph *Dag_16*, shown in Figure 1. From Figure 1, it is clear that *Dag_16* contains no directed cycles. You can verify this by appending the line of code 'DAG.is_directed_acyclic()' at the end of the program 'dacyclic16.sagews' and re-executing it. Upon execution of the modified program, you will get the output 'True', confirming that *Dag_16* is indeed acyclic.

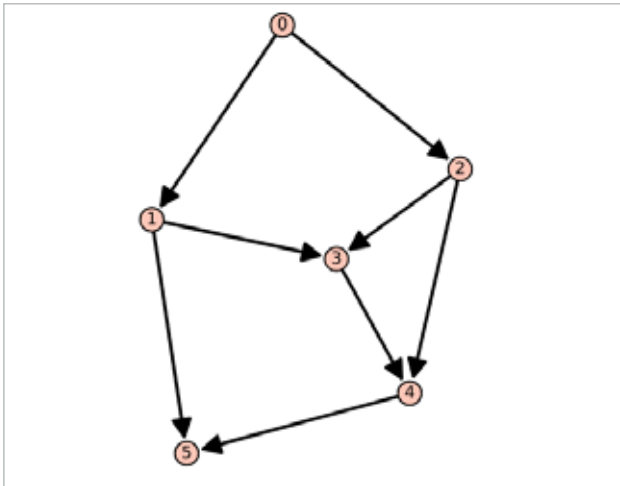


Figure 1: The directed acyclic graph *Dag_16*

An important task often performed in graph theory is obtaining the corresponding undirected graph from a directed graph. To do this, add the following lines of code at the end of the 'dacyclic16.sagews' program.

```
UG = DAG.to_undirected( )
UG.show( )
```

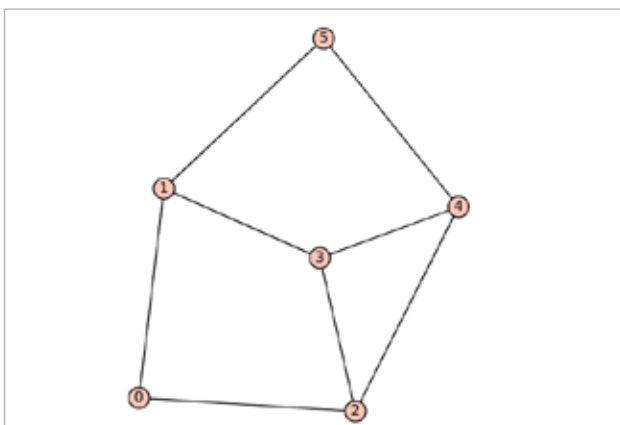


Figure 2: The undirected graph corresponding to *Dag_16*

Upon executing the modified program, the undirected graph corresponding to *Dag_16*, shown in Figure 2, is obtained.

Now, let us discuss weighted graphs (undirected), which we explored in the second article in this series (OSFY, February 2024). Operations research (OR) is the application of mathematical and analytical methods to optimise decisions and solve complex problems. For example, when dealing with distances in a graph, OR techniques can be used to find the shortest path between two vertices. Therefore, manipulating weighted graphs is an essential skill that every competent SageMath programmer should acquire. The program 'wgraph17.sagews', shown below, generates a weighted graph called *Wgraph_17* and attempts to print the length of the shortest path between vertices 0 and 1 (line numbers have been included for clarity in the explanation).

```
1. G = Graph({0: {2: 11}, 1: {2: 77}, 2: {3: 22},
2.       3: {0: 88, 1: 33}})
3. G.show(edge_labels=True)
4. shortest_path = G.shortest_path(0, 1)
5. shortest_path_len = G.shortest_path_length(0, 1)
6. print(shortest_path, '\n', shortest_path_len)
```

Now, let us go through the code line by line to understand how it works. Line 1 creates a weighted directed graph *G*. The graph is defined by specifying edges and their weights using a dictionary. For example, the code fragment '0: {2: 11}' means there is an edge from vertex 0 to vertex 2 with a weight of 11. Line 2 displays the graph with edge labels showing their weights. Line 3 computes the shortest path from vertex 0 to vertex 1 in the graph. The result is a list of vertices representing the path with the minimum total weight. Line 4 calculates the total length of the shortest path from vertex 0 to vertex 1. Finally, Line 5 prints the shortest path and its length.

Upon executing the program 'wgraph17.sagews' using CoCalc, we obtain the weighted graph *Wgraph_17*. The resulting graph, along with the (supposedly) shortest path and its length from vertex 0 to vertex 1, is shown in Figure 3.

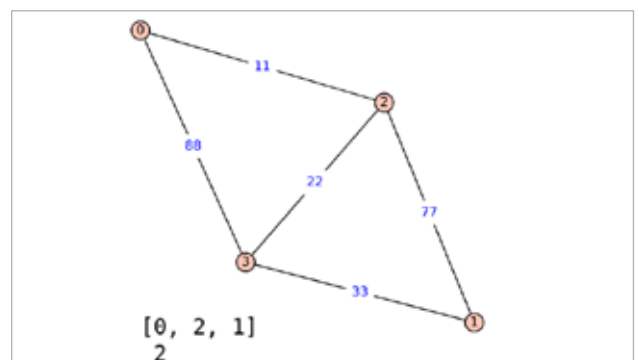


Figure 3: The weighted graph *Wgraph_17*